Session 7

State Management & Interactive Feature Design

Student Name: Guru K R

Project: Tip Time Calculator App (Jetpack Compose)

Course: Android / Jetpack Compose Development

Date: March 2026

Section 1: State Concepts Deep Dive

1.1 Key Definitions

State

State is any value in an application that can change over time and whose change must be reflected in the UI. In the Tip Time app, the bill amount typed by the user is state — when it changes, the displayed tip amount must update immediately. Compose observes state changes and automatically triggers recomposition of the affected parts of the UI.

Tip Time Example

```
// amountInput is state — it holds the current bill amount string
var amountInput by remember { mutableStateOf("") }
// When amountInput changes, the Tip Amount Text recomposes automatically
```

remember()

remember() stores a value across recompositions. Without it, every time the composable recomposes (for example when the keyboard appears), amountInput would reset to an empty string, wiping out whatever the user had typed. remember() tells Compose to retain the value in memory for the lifetime of the composable.

Tip Time Example

```
// Without remember, amountInput resets to "" on every recomposition
var amountInput by remember { mutableStateOf("") }
// The typed bill amount survives keyboard events and orientation changes
```

mutableStateOf()

mutableStateOf() wraps a value in Compose's snapshot state system, making it observable. When the user types in the Bill Amount field and onValueChange fires, amountInput is updated via mutableStateOf. Compose detects the write, marks the tip Text composable as stale, and schedules it for recomposition.

Tip Time Example

```
var amountInput by remember { mutableStateOf("") }
// Reading amountInput inside Text() registers that composable as an observer
// Writing amountInput (via onValueChange) triggers targeted recomposition
```

State Hoisting

State hoisting moves state out of a child composable and up to its parent, making the child stateless. In Tip Time, the EditNumberField composable does not own amountInput — it receives the current value and an onValueChange callback from TipTimeLayout. This makes EditNumberField reusable and independently testable.

Tip Time Example

```
// Parent owns the state
@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    EditNumberField(
        value = amountInput,
        onValueChange = { amountInput = it }   // callback hoisted to parent
    )
}

// Child is stateless — just displays and reports
@Composable
fun EditNumberField(value: String, onValueChange: (String) -> Unit, ...) {
    TextField(value = value, onValueChange = onValueChange)
}
```

Recomposition

Recomposition is Compose's process of re-running only the composables whose observed state has changed. In Tip Time, when the user types a new bill amount, only the tip result Text recomposes — the title Text and the TextField itself are skipped because their inputs have not changed. This makes the UI efficient and responsive.

Tip Time Example

```
val amount = amountInput.toDoubleOrNull() ?: 0.0
val tip    = calculateTip(amount)         // recalculates on every recomposition

// Only this Text recomposes when amountInput changes
Text(text = stringResource(R.string.tip_amount, tip))
```

1.2 Interaction Flow — Tip Time State Pipeline

The state flow in the Tip Time app follows a direct, single-input pipeline. The user taps the Bill Amount TextField and types a number. Every keystroke fires the onValueChange lambda, which writes the new string to amountInput. Because amountInput is a mutableStateOf delegate, Compose immediately marks the tip Text composable as stale. On the next frame, Compose re-executes TipTimeLayout: amountInput.toDoubleOrNull() converts the string to a number, calculateTip() multiplies it by the tip percentage (default 15%), NumberFormat.getCurrencyInstance().format() formats the result as a currency string, and the tip Text recomposes to display the updated value.

Flow: User types bill amount → onValueChange fires → amountInput state updates → amount and tip vals recalculate → tip Text recomposes → UI shows new tip amount.

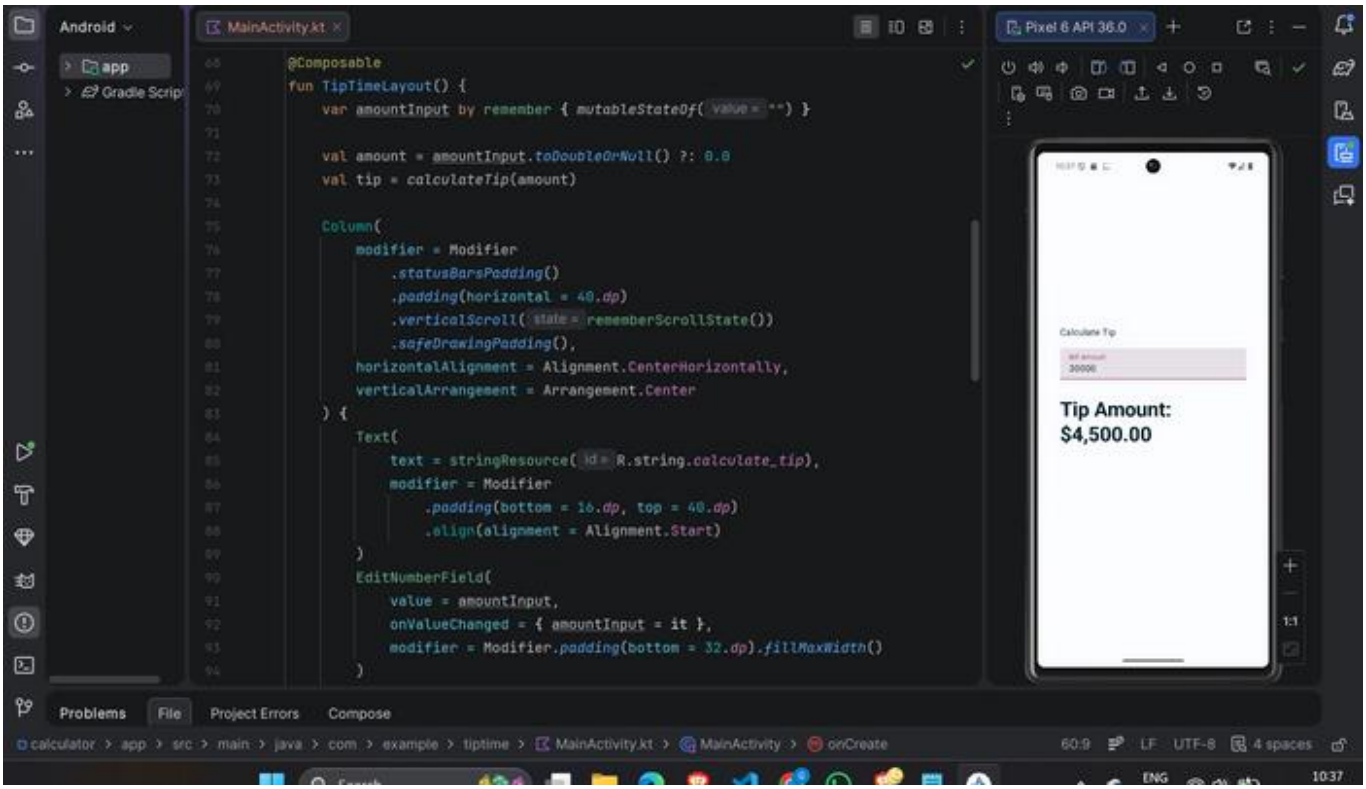

Fig 1a — TipTimeLayout composable: amountInput declared with remember { mutableStateOf("") }, derived tip calculation, and Pixel 6 preview showing Tip Amount: $4,500.00

```
                .align(alignment = Alignment.Start)
            )
            EditNumberField(
                value = amountInput,
                onValueChanged = { amountInput = it },
                modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
            )
            Text(
                text = stringResource( id = R.string.tip_amount,  ...formatArgs = tip),
                style = MaterialTheme.typography.displaySmall
            )
            Spacer(modifier = Modifier.height(150.dp))
        }
    }

    1 Usage
    @Composable
    fun EditNumberField(
        value: String,
        onValueChanged: (String) -> Unit,
        modifier: Modifier
```

Project Errors Compose

main > java > com > example > tiptime > MainActivity.kt > MainActivity > onCreate

Fig 1b — EditNumberField stateless composable receiving value and onValueChange from parent; tip_amount Text reading derived tip state

1.3 State Handling Strategy Table

UI Component	State Variable	Stateful / Stateless	Where State is Defined	Comments
Bill Amount TextField	amountInput (String)	Stateful	TipTimeLayout (parent)	Updates on every keystroke via onValueChange callback
Tip Amount Text	tip (derived val)	Stateless	Derived inside TipTimeLayout	Recalculates from amountInput on every recomposition; read-only display
EditNumberField	None (hoisted)	Stateless	Receives state from parent	Pure UI component; owns no state; reusable and independently testable

1.4 Reflection — State Hoisting and Correct State Management

State hoisting is essential in Jetpack Compose because it separates the concern of storing data from the concern of displaying it. In the Tip Time app, keeping amountInput inside TipTimeLayout rather than inside EditNumberField means the child composable is a pure, reusable UI element with no hidden internal logic — it can be previewed, tested, and reused in other screens without modification. If state were buried inside EditNumberField, the parent TipTimeLayout would have no way to read the bill amount and calculate the tip, making the feature impossible to implement without shared mutable state or callbacks that bypass Compose's model. Problems arise when state is not managed correctly: if amountInput were declared as a plain String variable without mutableStateOf, typing in the TextField would update the variable but Compose would never detect the change, so the tip display would always show $0.00. If amountInput were declared without remember, every keystroke would trigger a recomposition that resets amountInput to an empty string, making it impossible to type more than one character. Using the wrong type — for example, storing the amount as a Double instead of a String — would force premature conversion that loses the ability to represent partial inputs like '30.' as the user types. These pitfalls show that clear, correct state design is the foundation of a reliable Compose UI.

Section 2: Input, Validation & UI Feedback Plan

Feature Scenario: The Tip Time app allows users to enter a bill amount into a numeric text field. The app immediately calculates and displays the tip amount using a 15% default rate, formatting the result as a local currency string. The input must be validated so that empty or non-numeric values do not cause crashes.

2.1 Input Fields and State Variables

Input Component	State Variable	Validation Rule
Bill Amount TextField	amountInput: String	Must be convertible to a Double via toDoubleOrNull(). Non-numeric input is handled by the ?: 0.0 Elvis operator, defaulting the amount to 0.0 rather than crashing.
Tip Amount Text (display only)	tip: String (derived)	Read-only — derived from amountInput. No direct user input. Always shows a valid currency string because calculateTip() always returns a formatted result.

2.2 UI Feedback Plan

Scenario	UI Behaviour / Feedback
App launches / field is empty	Tip Amount displays $0.00. The TextField shows the placeholder label 'Bill Amount'. No error is shown — the empty state is a valid starting condition.
User types a valid number (e.g. 30000)	Tip Amount updates instantly on every keystroke. For 30000 at 15%, the display shows Tip Amount: $4,500.00 as seen in the screenshot.
User types a very large number (e.g. 555555)	The calculation still runs correctly. For 555555 at 15%, the display shows Tip Amount: $83,333.25. No overflow or formatting error occurs.
User clears the field	amountInput becomes an empty string. toDoubleOrNull() returns null, the Elvis operator substitutes 0.0, and the Tip Amount display returns to $0.00 without crashing.

2.3 Screen Layout Description

The Tip Time screen uses a vertically centred Column layout with the following structure from top to bottom:

- Title Text — 'Calculate Tip' heading displayed at the top of the content area, styled with padding on top and aligned to the start edge.
- Bill Amount TextField — An EditNumberField composable with a numeric keyboard (KeyboardType.Number). The field label reads 'Bill Amount' and the value is bound to the amountInput state via two-way data binding through the hoisted onValueChange callback.

- Tip Amount Text — A large, bold Text composable showing the formatted tip result (e.g. 'Tip Amount: $4,500.00'). It uses MaterialTheme.typography.displaySmall for prominence and is the primary output of the screen.
- Spacer — A 150.dp Spacer below the tip Text provides breathing room and prevents content from sitting too close to the bottom edge.

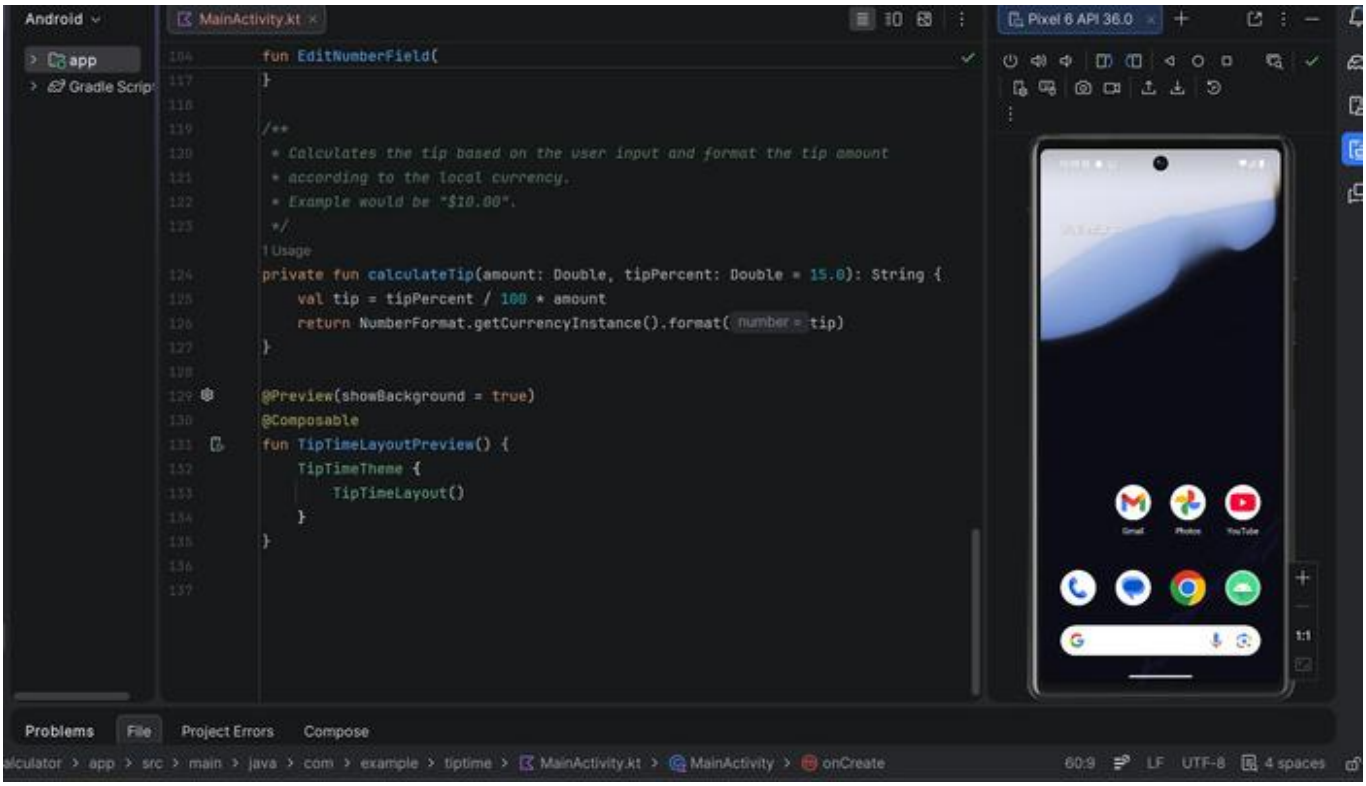

*Fig 2a — calculateTip() private function computing tip as (tipPercent / 100) * amount, formatted with NumberFormat.getCurrencyInstance()*

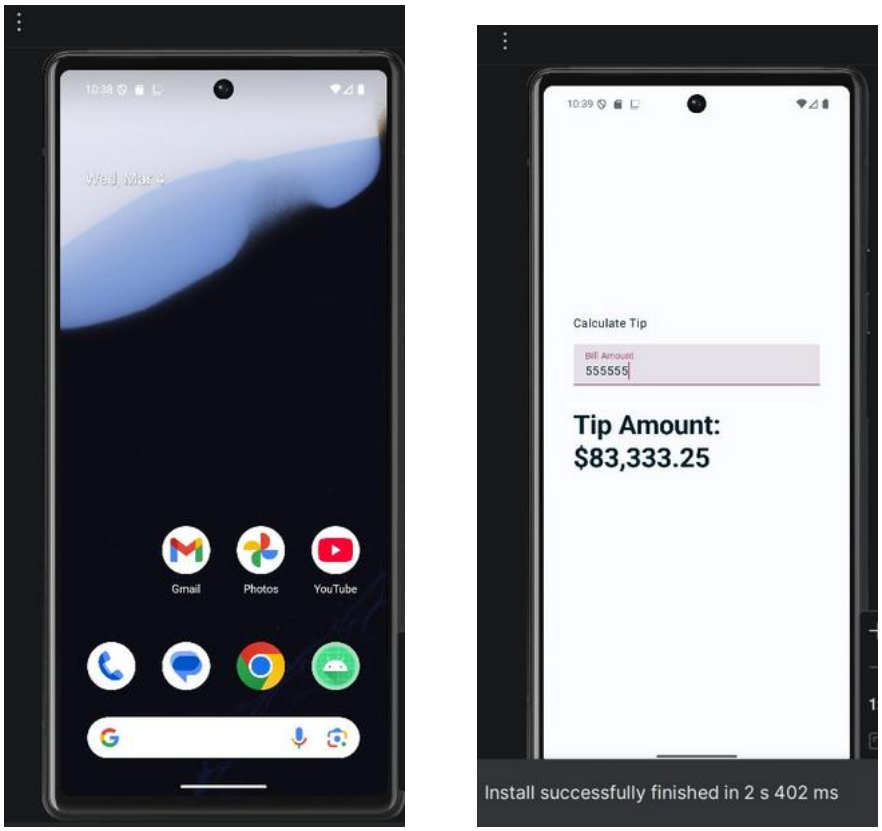

Fig 2b — Pixel 6 emulator: home screen after install (left) and Tip Time app running with bill amount 555555 showing Tip Amount: $83,333.25 (right)

2.4 Reflection — Importance of Validation and UI Feedback

Validation and UI feedback are critical for making the Tip Time app reliable and user-friendly. Because the bill amount is entered as a free-text string, the app must safely handle any input the user provides — including empty strings, partial decimals like '30.', and accidental non-numeric characters. Using toDoubleOrNull() with an Elvis operator fallback to 0.0 is a concise Kotlin idiom that prevents null pointer exceptions and app crashes without requiring explicit error handling code. Providing immediate, real-time feedback — where the tip amount updates on every single keystroke — gives users instant confidence that the app is working correctly and that their input has been understood. Displaying a formatted currency string like $4,500.00 rather than a raw double like 4500.0 makes the output immediately readable and trustworthy. In a production app, additional feedback such as disabling a 'Calculate' button when the field is empty, or showing a subtle error message for invalid input, would further improve the experience by preventing confusion about why no result is shown.

Section 3: Comprehensive Feature Design & Implementation Strategy

---

Feature Name: Tip Time — Tip Amount Calculator

Feature Description: The Tip Time app is a focused single-screen Jetpack Compose application that allows users to enter a bill amount and instantly see the calculated tip formatted as a local currency string. State is managed with remember and mutableStateOf, the input composable uses state hoisting, and the tip is derived directly from the state on every recomposition — demonstrating clean, reactive UI design.

3.1 Layout Description

- TipTimeLayout — the root @Composable that owns amountInput state and derives the tip value. Wraps everything in a Column with centred horizontal alignment.
- Title Text — displays the string resource R.string.calculate_tip at the top with padding.
- EditNumberField — a stateless @Composable wrapping a TextField. Receives value and onValueChange from the parent. Uses KeyboardType.Number to show a numeric keyboard.
- Tip Amount Text — reads the derived tip string and displays it with displaySmall typography. This is the primary output visible to the user.
- Spacer — 150.dp spacer below the result for comfortable layout on different screen sizes.

3.2 Component Actions Table

Component	Action / Behaviour	State Variable Affected
Bill Amount TextField	onValueChange fires on every keystroke; parent updates amountInput	amountInput (String)
Tip Amount Text	Reads derived tip val; recomposes automatically when amountInput changes	tip (derived — no direct mutation)
calculateTip() function	Called on every recomposition; computes tipPercent / 100 * amount and formats with NumberFormat.getCurrencyInstance()	No state — pure function

3.3 State Model

Variable	Type	Initial Value	Role
amountInput	String	""	Holds the raw bill amount string typed by the user
amount	Double	0.0	Derived: amountInput.toDoubleOrNull() ?: 0.0. Used by calculateTip()

tip	String	"$0.00"	Derived: result of calculateTip(amount). Displayed in Tip Amount Text

3.4 Actual Code from Project

```
@Composable
fun TipTimeLayout() {
    // ① State — holds the bill amount string across recompositions
    var amountInput by remember { mutableStateOf("") }

    // ② Derive Double from state safely (null-safe with Elvis operator)
    val amount = amountInput.toDoubleOrNull() ?: 0.0

    // ③ Derive formatted tip string — recalculates on every recomposition
    val tip = calculateTip(amount)

    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .verticalScroll(rememberScrollState())
            .safeDrawingPadding(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement  = Arrangement.Center
    ) {
        // ④ Title
        Text(
            text     = stringResource(R.string.calculate  tip),
            modifier = Modifier.padding(bottom = 16.dp, top = 40.dp)
                            .align(Alignment.Start)
        )

        // ⑤ Hoisted input field — state lives in parent, not here
        EditNumberField(
            value          = amountInput,
            onValueChange  = { amountInput = it },
            modifier       = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )

        // ⑥ Tip result — recomposes when tip changes
        Text(
            text  = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}

// ⑦ Stateless child composable — state hoisted to TipTimeLayout
@Composable
fun EditNumberField(value: String, onValueChange: (String) -> Unit, modifier: Modifier) {
    TextField(value = value, onValueChange = onValueChange, modifier = modifier)
}

// ⑧ Pure calculation function — no state, always same output for same input
```

```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0): String {
    val tip = tipPercent / 100 * amount
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

3.5 Test Cases

Test Case 1 — Standard Tip Calculation

Input: User types 30000 in the Bill Amount field.

Expected Result: Tip Amount displays '$4,500.00' (30000 × 15% = 4500, formatted as currency). Matches the screenshot in Fig 1a.

Test Case 2 — Large Bill Amount

Input: User types 555555 in the Bill Amount field.

Expected Result: Tip Amount displays '$83,333.25' (555555 × 15% = 83333.25). No overflow or formatting error. Matches the screenshot in Fig 2b.

Test Case 3 — Empty Field

Input: User clears the Bill Amount field (amountInput = "").

Expected Result: toDoubleOrNull() returns null, Elvis operator substitutes 0.0, calculateTip(0.0) returns '$0.00'. App does not crash. Tip Amount shows $0.00.

3.6 Scaling Considerations

Adjustable Tip Percentage

A Slider or a set of percentage buttons (10%, 15%, 20%) can be added as a second state variable var tipPercent by remember { mutableStateOf(15.0) }. The calculateTip() function already accepts tipPercent as a parameter, so wiring up a Slider requires only passing the new value on change.

Round Up Toggle

A Switch composable bound to a var roundUp by remember { mutableStateOf(false) } state variable can pass a rounding flag to calculateTip(). When enabled, kotlin.math.ceil() rounds the tip up to the nearest dollar, a common real-world requirement.

Split Between Guests

A number-of-guests field can be added as additional state. The per-person amount is derived as tip / guests, displayed in a third Text composable alongside the total tip, extending the app into a full bill-splitting tool without changing the core state architecture.

3.7 Reflection — Challenges and Compose's Role

Planning the Tip Time app state was simpler than a multi-input calculator, but it still surfaced important Jetpack Compose principles. The main challenge was understanding why amountInput must be stored as a

String rather than a Double: the TextField works with raw user input character by character, so storing it as a Double would force premature conversion and break the ability to represent intermediate states like an empty field or a partial decimal such as '30.'. Letting the conversion happen at the point of use with toDoubleOrNull() and a safe fallback is the correct Compose pattern. Another challenge was grasping state hoisting: it feels counterintuitive at first to define amountInput in the parent TipTimeLayout rather than inside EditNumberField where the user types, but doing so is what allows TipTimeLayout to read the value and calculate the tip. Jetpack Compose made the reactive data flow straightforward once the state model was correct: because tip is a val derived directly from amountInput on every recomposition, there is no explicit logic needed to 'push' updates to the tip display — Compose handles all of that automatically. This demonstrates the power of Compose's declarative model: describe what the UI should look like for a given state, define the state clearly, and Compose takes care of keeping them in sync.